

IC526 Advanced Compilers

Lec 02: OCaml Basics

Yoonseung Kim

DGIST EECS

Fall 2026

Why OCaml?

- ~~• Because the professor likes it.~~
- ~~• Because functional programming is superior.~~
- Functional programming with algebraic data types fit compiler development.
 - Language syntax, types, analysis, ...
- Real compilers & code analyzers use it.
 - CompCert, Infer, Rust (early)
- Learning a new language is part of the point.
 - You'll see what a high-level language features can do for you.

Why OCaml?

- Algebraic data types + pattern matching = natural fit for compilers.

[expr]

- 3
- 4
- 12
- 3 + 12
- (3 + 12) + 4
- ...



```
type expr = Num of int | Add of expr * expr
```

```
let rec eval e =  
  match e with  
  | Num n -> n  
  | Add (e1, e2) -> eval e1 + eval e2
```

The core pattern you'll use throughout this course

Setup

- Follow the setup instructions on the course website.
 - Package setup – `helps/Setup.md`
 - IDE setup – `helps/Editor.md`
 - AI tools will help you troubleshoot.
- Bring your laptop for in-class exercises.

Run OCaml

- Interactive mode

```
$ utop
```

- To run a single file (for practice)

```
$ ocaml file.ml # uses interpreter
```

```
$ ocamlc file.ml -o file && ./file # uses compiler
```

- **To build a project** – use *Dune*

```
$ dune init proj my_proj # generates new project named “my_proj”
```

```
$ dune exec bin/main.exe # build and run; for build-only: dune build
```

Hello, World!

`$ dune init proj my_proj` `# create a test project 'my_proj'`

`$ code .` `# if you use VS Code`
 `# don't miss the dot [.]`

- Open bin/main.ml in VSCode

```
let () = print_endline "Hello, World!"
```

- Terminal → New Terminal
`$ dune exec bin/main.exe`

Basic Features

Global *Let* Bindings

- An .ml file contains a sequence of **bindings**

```
let my_val = 2 + 3

let add n m = if n = 0 then m else n + m

let () = Printf.printf "Add my_val and 3: %d\n" (add my_val 3)
```

1. RHS expression evaluates to a value
2. The LHS variable *is bound to the value*

Printf.printf : A builtin *effectful* function that returns ***unit*** [()]

Forget about **memory & commands!**
Only ***expressions*** and ***global bindings***.

Local *Let* Bindings and Scopes

- `[let x = e1 in e2]` locally binds `[x]` to the value of `[e1]` in `[e2]`.
- `[x]` is only visible in `[e2]`: “[`e2`] is the scope of `[x]`”.

```
let x = 10                                (* global binding *)

let rec fib n =                           (* recursive function needs [rec] *)
  if n == 0 then 0                         (* 'if' expression *)
  else if n == 1 then 1                   (* just a nested if *)
  else
    let x = fib (n - 2) in                (* local binding - shadows global [x] *)
    let y = fib (n - 1) in                (* local binding (nested) *)
    x + y

let () = Printf.printf "fib x = %d\n" (fib x)    (* fib(10) = 55 *)
```

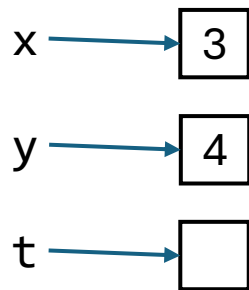
Binding ≠ Storing in Memory

C++ (Storing in memory)

- A variable is mapped to a cell's addr

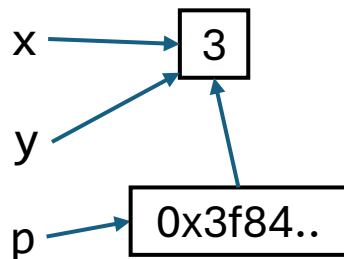
Update is allowed

```
// x = 3, y = 4
t = x; x = y; y = t;
// x = 4, y = 3
```



Multiple variables may point the same cell

```
// x = 3
int &y = x;
int *p = &x;
y = 4 // x = 4
*p = 5 // x = 5
```

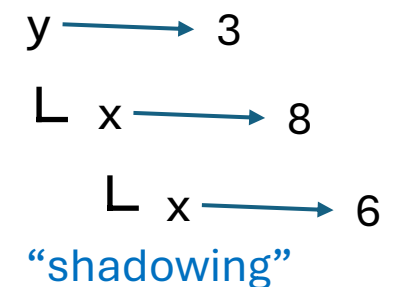


OCaml (Binding)

- A variable is mapped to a value

No updates – let only generates a new scope

```
let y = 3 in
let x = y + 5 in
(let x = y * 2 in
  x + 2) * x
```



Why this design?

```
(fib x) + ..
.. (* no let x *)
(fib x) * ..
```

Not true in C++, in general
(why?)

```
let t1 = fib x in
t1 + ..
..
t1 * ..
```

Note: The concepts of shadowing & scope exist in C++, too; but their usage are more critical in Ocaml.

Scope Practice: Nested Let Bindings

```
let x = 5
```

```
let () =  
  let y =  
    let x = x + 1 in  
    (let x = x * 3 in  
      x + 2) + x  
  in  
  Printf.printf "%d\n" (x + y)
```

(Answer: 31)

Loops? Use Recursion!

C++

- Implement “is_prime”

```
bool is_prime(int n) {  
    for (int i=2; i < n; ++i)  
        if (n % i == 0)  
            return false;  
  
    return true;  
}
```

OCaml

```
let rec is_prime_i i n =  
    if i < n then  
        if (n mod i == 0) then false  
        else is_prime_i (i+1) n  
    else true  
  
let is_prime n =  
    is_prime_i 2 n
```

It's a Typed Language

- Every expression has a *type*, determined at **compile time**.
 - *Type errors* are caught **before** running the program.
- Types can be **inferred** — in many cases, you may omit them.
 - Still, type annotations help **improve readability** and **reduce errors**.

```
let x : int = 10      (* x : int *)
let x2 = 10           (* x2 : A => Since 10 : int, A = int *)

let double (n : int) : int = (* double : int -> int *)
  let y : int = n * 2 in
  y

let double2 n =        (* double2 : A -> B, and n : A *)
  let y = n * 2 in      (* 2 is int => A = int, therefore, y : int *)
  y                    (* y : int => B = int *)
```

It's a Functional Language

- Functions are **first-class values** — they can be passed around like any other value.

```
let apply_twice (f: int -> int) (x: int) : int =    (* function parameter *)
  f (f x)

let add (n m: int) : int = n + m

let () =
  let () = Printf.printf "double (double 2) = %d\n" (apply_twice double 2) in
  let add3 k = k + 3 in                                (* local function *)
  let () = Printf.printf "add3 (add3 5) = %d\n" (apply_twice add3 5) in
  let () = Printf.printf "add2 (add2 5) = %d\n" (apply_twice (add 2) 5) in
                                                    (* partial application *)
  Printf.printf "add2 (add2 5) = %d\n" (apply_twice (fun x -> x + 2) 5)
                                                    (* anonymous function *)
```

[;] : Syntactic Sugar

- `[e1; e2]` is a shorthand for `[let () = e1 in e2]`

```
let apply_twice (f: int -> int) (x: int) : int = (* function parameter *)
  f (f x)

let add : int -> int -> int = fun n m -> n + m      (* the same definition *)

let () =
  let add3 k = k + 3 in
  Printf.printf "double (double 2) = %d\n" (apply_twice double 2);
  Printf.printf "add3 (add3 5) = %d\n" (apply_twice add3 5);
  Printf.printf "add2 (add2 5) = %d\n" (apply_twice (add 2) 5);
  Printf.printf "add2 (add2 5) = %d\n" (apply_twice (fun x -> x + 2) 5)
```

ADT and Pattern Matching

Algebraic Data Types (ADT)

type name

3 constructors

```
type binop = Add | Sub | Mul

type expr =
  | Num of int
  | BinOp of binop * expr * expr

let num3 : expr = Num 3
let sub_expr1 = BinOp (Sub, num3, Num 2)
```

- Each case has a *constructor* of the type.
- Constructors may carry data.

Pattern Matching – How to “Use” ADTs

```
type binop = Add | Sub | Mul

type expr =
  | Num of int
  | BinOp of binop * expr * expr

let rec eval e =
  match e with
  | Num n -> n
  | BinOp (op, e1, e2) ->
    match op with
    | Add -> eval e1 + eval e2
    | Sub -> eval e1 - eval e2
    | Mul -> eval e1 * eval e2
```

- Exhaustiveness check — **compiler warns** if you miss a case.

Pattern Matching – How to “Use” ADTs

```
type binop = Add | Sub | Mul

type expr =
  | Num of int
  | BinOp of binop * expr * expr

let rec eval' e =
  match e with
  | Num n -> n
  | BinOp (Add, e1, e2) -> eval' e1 + eval' e2
  | BinOp (Sub, e1, e2) -> eval' e1 - eval' e2
  | BinOp (Mul, e1, e2) -> eval' e1 * eval' e2
```

- Patterns can be nested.

List: A Widely-Used ADT for Sequences

```
let xs : int list = [1; 2; 3]
let es : expr list = [Num 3; BinOp (Mul, Num 1, Num 2)]

let ys = 0 :: xs          (* [0; 1; 2; 3] *)
let zs = xs @ [4; 5]      (* [1; 2; 3; 4; 5] *)

let ys = List.map double xs (* [2; 4; 6] *)
```

```
type 'a list =
| []
| (::) of 'a * 'a list

let rec map (f: 'a -> 'b) (l: 'a list) : 'b list =
  match l with
  | [] -> []
  | a :: l' -> (f a) :: (map f l')
```

Option Type: Representing Failures

```
type 'a option =  
  | None  
  | Some of 'a  
  
let rec nth (l: 'a list) (n: int) : 'a option =  
  match l with  
  | [] -> None  
  | h :: t -> if n = 0 then Some h else nth t (n-1)
```

Ready To Go!

Exercises: HW0

- Create an ml project using Dune.
- Open `hw_files/hw0.ml` in the course website.
- Copy and past the content of `hw0.ml` into your `main.ml`
- Try now!

Linking multiple .ml Files (by Dune)

def.ml

```
type binop = Add | Sub | Mul

type expr =
  | Num of int
  | BinOp of binop * expr * expr

let rec eval e =
  match e with
  | Num n -> n
  | BinOp (op, e1, e2) ->
    match op with
    | Add -> eval e1 + eval e2
    | Sub -> eval e1 - eval e2
    | Mul -> eval e1 * eval e2
```

main.ml

```
let expr : Def.expr = Def.Num 3

let result = Def.eval expr
```

Or,

main.ml

```
open Def

let expr : expr = Num 3

let result = eval expr
```


More to Learn!

- Exceptions
- Records
- References (not necessary for this course)

Learn By Exercise: Assignment #1

- Build a tiny compiler: **arithmetic expressions** → **stack machine**.

Source language

```
1 + 2 * 3
```

Target: a stack machine

```
PUSH 1; PUSH 2; PUSH 3; MUL; ADD
```

- (1) Write a compiler
- (2) Write a “simulator” – an interpreter of the stack language
- (3) Write an interpreter using the option type
- (4) Write an interpreter using exceptions

How To Do Assignments

- Check the notice – leave your ID and name until next Monday.
 - You will be given a private GitHub repository.
 - The repository contains the skeleton project.
- For each assignment, the specification document will be posted on the course website.
 - Check the course website.
 - Push your homework in time.